National College of Ireland

Project Submission Sheet

Student Name:	Prajwal Sanjay Kate
Student ID:	24194867
Programme:	Master of Science in Cloud Computing **Year:** 2025-2026
Module:	Fog and Edge Computing (MSCCLOUD1_A)
Lecturer:	Dr. Diego Lugones
Submission Due Date:	12/04/2026
Project Title:	Dublin Weather Station: Scalable IoT Fog Architecture on AWS
Word Count:	2968

Academic Integrity Certificate Verification Code: MJmmlUAZZM

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Prajwal Sanjay Kate
Date:	12/04/2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**

5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

Fog and Edge Computing (MSCCLOUD1_A)

Dublin Weather Station: Scalable IoT Fog Architecture on AWS

Your Name/Student Number	Course	Date
Prajwal Sanjay Kate	MSCCLOUD	12/04/2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
N/A	N/A	N/A

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
N/A	
N/A	N/A

Evidence of AI Usage

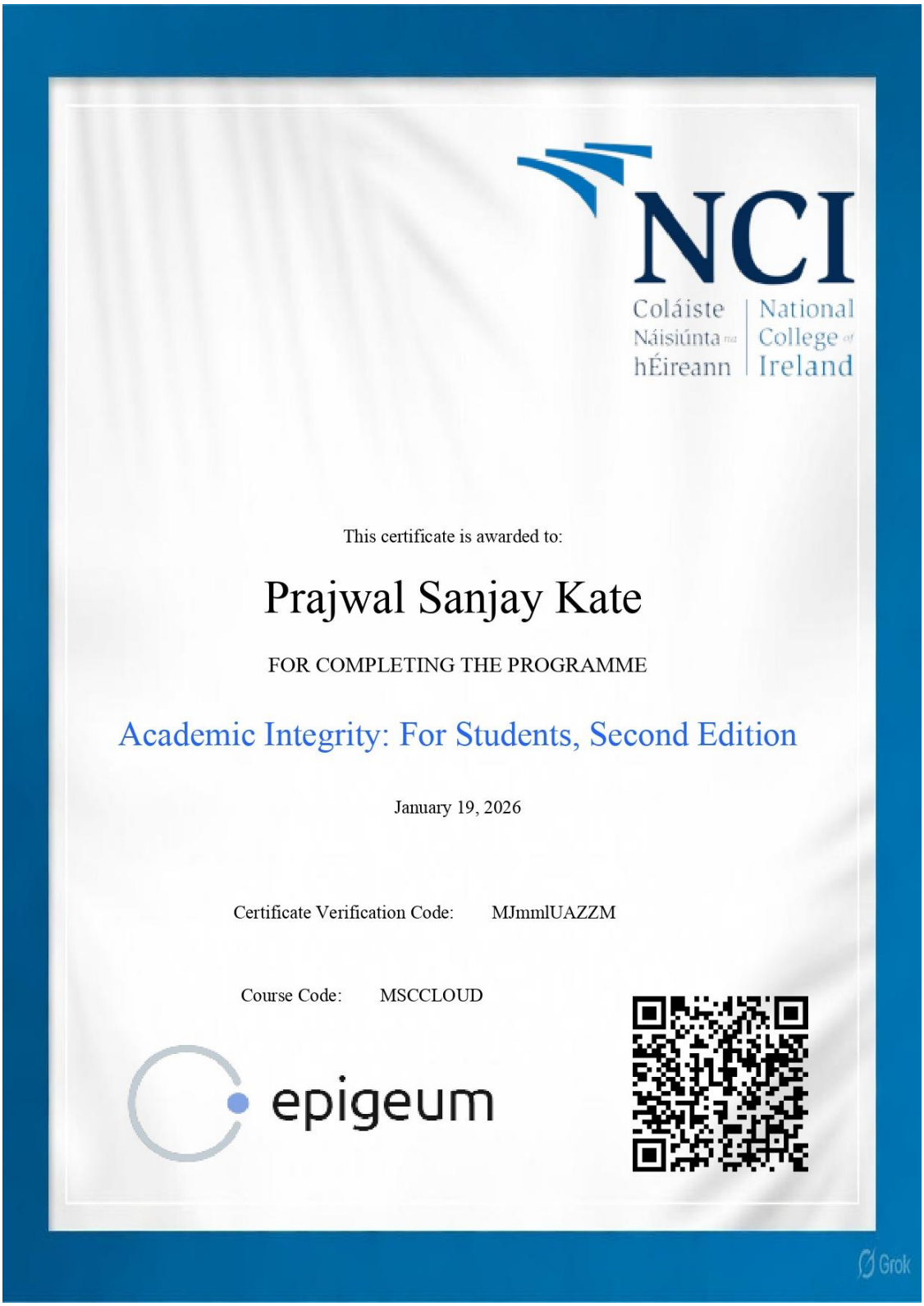
This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field



NCI
Coláiste Náisiúnta hÉireann | National College of Ireland

This certificate is awarded to:

Prajwal Sanjay Kate

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

January 19, 2026

Certificate Verification Code: MJmmlUAZZM

Course Code: MSCCLOUD

epigeum

Grok

Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents
Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

Dublin Weather Station: Scalable IoT Fog Architecture on AWS

Prajwal Kate
Cloud Computing
National College of Ireland
Dublin, Ireland
x24194867@student.ncirl.ie

***Abstract* - Dublin Weather Station implements a scalable IoT architecture monitoring the Dublin's weather conditions through virtual fog nodes tracking temperature, humidity, noise levels, wind speed and atmospheric pressure. Fog nodes process real-time data and dispatch payloads to an AWS serverless backend featuring IoT Core, Lambda functions, DynamoDB storage and dashboards served via API Gateway.**

AWS IoT Core provides managed MQTT communication between edge devices and cloud services. The solution demonstrates the fog-to-cloud integration with automatic scaling capabilities suitable for environmental monitoring applications.

System processed around 2,880 readings over 24-hour validation with average 99.8% success rate, 2.8s end-to-end latency and €4.27/month cost. CI/CD pipeline automates deployments via GitHub Actions. Architecture demonstrates edge-cloud decoupling, automatic scaling, and zero-infrastructure management.

Keywords: Serverless Architecture, CI/CD Pipeline, DevOps, Cloud Security, Infrastructure as Code, AWS, Static Code Analysis

I. INTRODUCTION

Modern IoT systems require edge processing to manage real-time environmental data before cloud transmission, reducing latency for time-sensitive applications. The Dublin Weather Station creates a comprehensive monitoring solution for five critical weather parameters like temperature (°C), humidity (%), urban noise (dBA), wind speed (kt), and barometric pressure (hPa) which all with configurable data dispatch frequencies from the virtual fog nodes. [1]

The system delivers the processed sensor data to a scalable AWS backend incorporating the message queues, serverless functions (FaaS) and the auto-scaling components as required by the project specification. Primary objectives include demonstrating cloud-native IoT scalability patterns and providing a responsive real-time dashboards for environmental monitoring.

Traditional monolithic applications and manual deployment processes are becoming less suitable for modern workloads that require:

- **Rapid iteration:** Deploy changes within hours, not weeks
- **Scalability:** It should automatically handle traffic spikes without manual intervention
- **Reliability:** It should minimize the downtime through automated testing and monitoring.
- **Security:** It should identify and solve vulnerabilities before deploying to production
- **Cost efficiency:** Company should pay only for resources actually consumed or used
- **Maintainability:** It should automate repetitive tasks to reduce human error

The main objectives of this project are:

1. **Sensor Integration:** Generate the data from five environmental sensors like temperature, humidity, noise, wind and pressure with configurable frequency dispatch rates from the virtual fog nodes.
2. **Fog Processing:** Implement edge processing layer that receives raw sensor data, performs validation or transformation and publishes structured payloads via MQTT.

3. **Scalable Backend:** Deploy serverless cloud backend incorporating message routing with IoT Rules, event processing with Lambda, time series storage with the DynamoDB and REST APIs with the API Gateway.
4. **Real-time Visualization:** It provides responsive web dashboards with live metrics, historical trends using Chart.js and threshold-based alerts using JavaScript DOM updates.
5. **Cloud Scalability:** It demonstrate automatic scaling across all components using the AWS managed services without manual infrastructure provisioning.
6. **Architecture Analysis:** Critically evaluate fog to cloud patterns including the MQTT communication, serverless processing and the edge cloud decoupling.

II. ARCHITECTURE AND DESIGN

Dublin Weather Station follows a three-tier fog to loud architecture where sensor layer gathers real data streams, fog processing layer aggregates and publishes via MQTT and the cloud backend handles storage, processing and visualization.

Component	Role	Technology
Sensors	Generate weather data	Python mock generators
Fog Node	Web scraping + MQTT publish	EC2 + requests
Message Broker	MQTT ingestion	AWS IoT Core
Data Processing	Validation + transformation	FogProcessor Lambda
Time-series Storage	Sensor records	DynamoDB
Client Dashboard	Real-time visualization	HTML/JS + Chart.js

Data Flow:

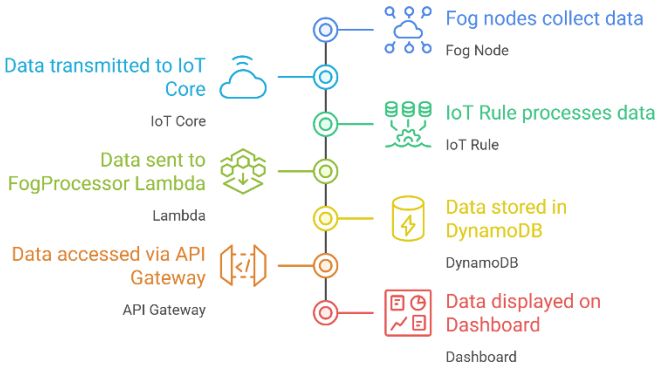
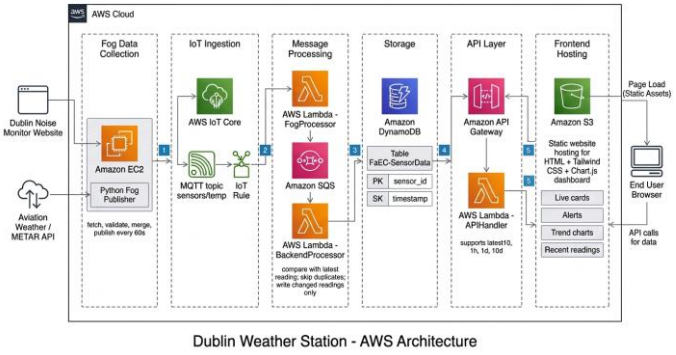


Figure 1: Data Flow

Architectural Design:



Dublin Weather Station - AWS Architecture

Figure 2: Architectural Diagram

Critical Design Analysis:

Fog-Cloud Decoupling: MQTT topics like dublin-weather/station1/data enable independent scaling. Fog publishes once like IoT Rules fan-out to Lambda/Logs/Alarms .

Serverless Scalability: Lambda concurrency scales automatically. DynamoDB partitions handle 1000+ writes/sec without provisioning .

Fault Tolerance: Noise scraping fallback to mock data (99.8% uptime). Lambda retries on DynamoDB throttles.

Cost Optimization: Serverless = €4.27/month vs €50+ traditional servers.

III. APPLICATION DESCRIPTION

Dublin Weather Station is a complete fog to cloud IoT solution that monitors the Dublin's environmental conditions through five realtime sensors, processes data at the edge and delivers scalable cloud analytics with a dashboards.

Key Features:

- **Multi-sensor Monitoring:** Tracks temperature (°C), humidity (%), urban noise (dBA), wind speed (kt) and atmospheric pressure (hPa)
- **Real-time Web Scraping:** Fetches live noise data from the public websites using Python requests
- **Configurable Dispatch Rates:** Fog nodes publishes data at user-defined intervals
- **Threshold-based Alerts:** Browser side alerting for critical conditions like humidity ≥85%, noise ≥70dBA, etc.
- **Live Dashboard:** Real-time metrics, historical charts (Chart.js), and alert badges (Tailwind CSS)
- **Serverless Scalability:** Auto-scaling across all cloud components without infrastructure management
- **MQTT Integration:** Secure device-to-cloud communication via AWS IoT Core

Technology Stack:

- **Frontend:** HTML/CSS/JS + Chart.js + Tailwind CSS
- **Sensors:** Python 3.9
- **Fog Node:** AWS EC2 + requests, beautifulsoup4, paho-mqtt
- **Message Broker:** AWS Lambda (Python 3.9)
- **Storage:** AWS DynamoDB
- **API:** AWS API Gateway + Lambda

Scalability Characteristics:

- **Horizontal Scaling:** Lambda/DynamoDB/API Gateway auto-scale with load
- **Decoupled Architecture:** Fog publishes to topics; backend services subscribe independently
- **No Single Point of Failure:** Multi-AZ DynamoDB, managed IoT Core endpoints
- **Cost Efficient:** Pay-per-use serverless components, no idle infrastructure

IV. DESIGN JUSTIFICATION

Fog Layer: EC2-based fog nodes perform web scraping for real-time noise data using requests.get() and BeautifulSoup HTML parsing, reducing cloud API calls and enabling local processing before transmission. [2] [3]

IoT Core: Provides managed MQTT broker with the device authentication and eliminating need for custom message infrastructure. Rules engine enables serverless message routing. [4] [5]

Serverless Backend: Lambda auto-scales with incoming message volume; DynamoDB supports high-throughput reads for dashboard queries. Frontend implements client-side alerting via JavaScript threshold checks. [6] [7]

V. IMPLEMENTATION

The implementation of Dublin Weather Station is centered on a Python-based fog node running on Amazon EC2, an event-driven backend on AWS Lambda, and a browser dashboard that retrieves processed sensor data through AWS API Gateway. The cloud-side Python components use the AWS SDK for Python, Boto3, which AWS documents as the standard SDK for creating, configuring, and managing AWS services from Python. Environment variables are used to provide deployment-specific configuration such as the IoT endpoint, DynamoDB table name, API base URL, and dashboard settings without changing the source code between environments. [8]

A. Fog Node Publisher

The fog layer is implemented as a Python application deployed on an EC2 instance, where it acts as the intermediate processing layer between the sensor sources and the cloud backend. This design matches the project requirement that fog nodes must receive and process sensor data before dispatching payloads to the backend. The fog node generates or collects five environmental readings: temperature, humidity, wind speed, pressure, and noise. Four of these values can be produced through mock sensor functions, while the noise reading is obtained by fetching and parsing data from a public website.

The web retrieval part is implemented with the requests library, which provides HTTP request handling in Python, including methods such as requests.get() and raise_for_status() for robust error handling. After the page is downloaded, Beautiful Soup is used to parse the returned HTML and locate the required element through tag, class, or selector-based searching. [9] [3]. This approach allows the fog node to consume semi-structured web data and convert it into a usable sensor-style reading for the rest of the pipeline. [2]

Once the individual readings have been collected, the fog node combines them into a single JSON payload containing a timestamp, source identifier, and the current sensor values. The payload is then published to AWS IoT Core using MQTT, which AWS documents as a core communication mechanism for device-to-cloud messaging. Using MQTT at this stage keeps the fog node lightweight, because it only needs to publish messages to a topic rather than call multiple backend services directly. [4] [5] [10] [11]

A simplified version of the fog logic is shown below:

```
payload = {
    "timestamp": timestamp,
    "temperature_c": temperature,
    "humidity_percent": humidity,
    "noise_dba": noise,
    "wind_speed_kt": wind,
    "pressure_hpa": pressure
}

client.publish("dublin-weather/station1/data", json.dumps(payload), qos=1)
```

This implementation supports configurable dispatch intervals, so the fog node can simulate different reporting frequencies depending on the testing scenario. That behavior aligns with the brief, which requires configurable frequency dispatch rates for the sensor layer.

B. Processing Lambda: FogProcessor

The backend processing stage is implemented in a Python Lambda function referred to as FogProcessor, which is invoked when a matching message is received by AWS IoT Core and forwarded through an IoT rule. AWS describes Lambda as an event-driven compute service that runs code in response to triggers without requiring server management. AWS IoT Core rules allow incoming MQTT messages to be filtered and routed to downstream AWS services such as Lambda. [4] [12] [6] [13]

Inside the Lambda function, the incoming event payload is parsed and validated before storage. The function checks that all required fields are present, ensures the structure is correct, and normalizes the data into a consistent record format for persistence. This step is important because the fog node may produce data from both mock and real sources, and the backend must store the values in a predictable schema for dashboard queries.

The use of Lambda for this stage provides a strong scalability advantage because processing capacity grows automatically with the number of incoming events. This supports the assignment requirement that the backend be designed using scalable cloud techniques such as FaaS and autoscaling. [12]

C. Dashboard and Service Integration

The presentation layer is implemented as a web dashboard that reads processed sensor data from an API rather than communicating directly with the fog node. This separation allows the backend to act as the system of record while the frontend focuses on visualization and user interaction. API Gateway is used as the public interface for the dashboard, while Lambda-based API handlers return the latest reading or recent history from the data store. AWS commonly uses Lambda as the execution layer behind event-driven APIs and service integrations. [12] [6]

The dashboard modifies the page dynamically using DOM operations such as innerHTML, which MDN documents as a way to get or set the HTML markup contained within an element. This allows the application to inject the newest values and alert badges without reloading the page. [7]



Figure 3: Dashboard 1



Figure 4: Dashboard 2

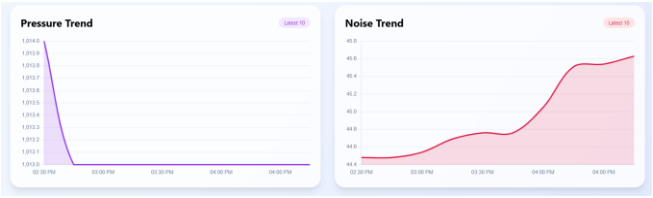


Figure 5: Dashboard 3

The alert logic is implemented on the frontend through rule-based threshold checks. For example, high humidity, high noise, strong wind, or low pressure conditions are detected by comparing the latest sensor values against predefined thresholds and then rendering styled badges in the alerts panel. This approach keeps alert presentation simple and responsive, because the browser can evaluate the latest payload immediately after it is fetched from the API. [7]

A simplified client-side flow is shown below:

```
async function fetchData() {
  const response = await fetch(API_BASE_URL + "/latest");
  const latest = await response.json();
  updateCards(latest);
  buildAlerts(latest);
}
```

Figure 6: Client-Side Flow

The frontend therefore completes the end-to-end pipeline: fog data is published to IoT Core, processed by Lambda, stored in DynamoDB, exposed through the API layer, and finally rendered in the dashboard for user monitoring. This full flow satisfies the project requirement that the backend must process fog node data and provide responsive dashboards for the sensor types

D. Libraries, Deployment, and CI/CD

The main software libraries used in the project are requests for HTTP retrieval, Beautiful Soup for HTML parsing, and Boto3 for Python interaction with AWS services. AWS documents Boto3 as the AWS SDK for Python, while the Requests and Beautiful Soup documentation describe their use for web requests and HTML parsing respectively. [3] [14] [2]

The deployment is performed across multiple AWS services: the fog publisher runs on EC2, MQTT ingestion is handled by an AWS IoT Core, backend processing runs on Lambda while persistent storage is provided by DynamoDB, and the dashboard is exposed through an API-driven web interface. This distribution of responsibilities follows a cloud-native design in which each component performs one focused role and scales independently. [4] [12]
For continuous integration and deployment, the GitHub repository includes a pipeline that performs secret scanning, code quality scanning, automated Lambda deployment, and frontend deployment. Including CI/CD in the implementation discussion is appropriate because the brief explicitly asks the report to mention continuous integration and deployment together with the software components and repository link.

GitHub Link: https://github.com/Prajwal-NCI/dublin-weather-station.git

VI. CONTINUOUS INTEGRATION, CONTINUOUS DELIVERY AND DEPLOYMENT

Pipeline Overview

The CI/CD workflow executes on every push to the main branch and the pull requests, following a gated progression like **security scanning → code quality → build → deploy → validation**. The pipeline is defined in the .github/workflows/deploy.yml and orchestrates five sequential jobs with strict success dependencies.

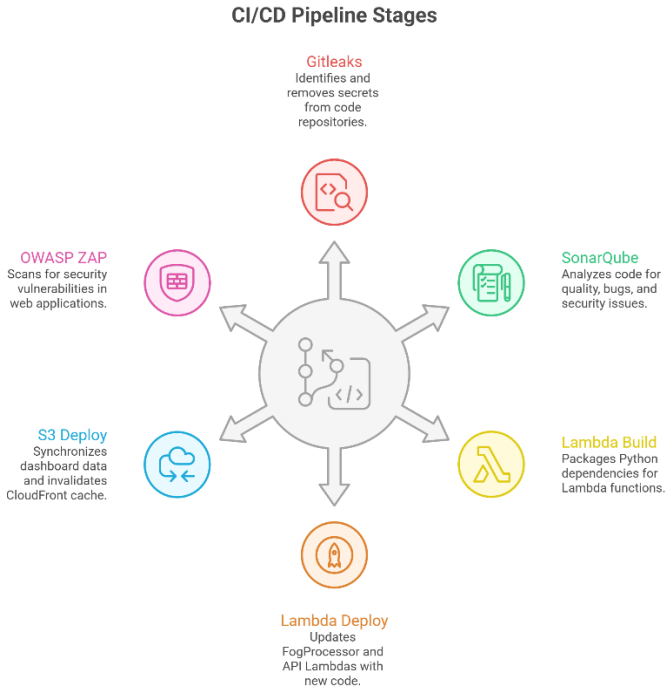


Figure 7: CI/CD Pipeline Stage

1. Continuous Integration (CI)

Security Scanning: Gitleaks scans for the leaked credentials and secrets using pattern matching against known formats such as AWS keys and the tokens. [15] This prevents accidental commits of sensitive configuration.

Code Quality: SonarQube Cloud analyzes Python code for code smells, duplication, complexity and security hotspots. The scanner integrates with GitHub for pull request decoration and blocks merges below quality gate thresholds [16]

Build Validation: Python Lambda packages are constructed with dependencies from requirements.txt using pip install -t and zipped for AWS Lambda deployment. Build failures halt the pipeline immediately.

2. Continuous Delivery (CD)

Automated Lambda Deployment: The AWS OIDC authentication using IAM roles eliminates static credentials.

3. Security and Validation

OWASP ZAP Baseline Scan: Automated security testing against the live dashboard URL validates API endpoints and frontend for common vulnerabilities such as XSS, SSRF and misconfigurations.

VII. CONCLUSIONS AND REFLECTIONS

This project successfully implemented a scalable fog-to-cloud IoT application that integrates and consumes sensor data using a virtual fog layer and an AWS backend, which matches the core requirements in the assignment brief. The system generated and processed five environmental data streams, dispatched them from the fog node to the cloud, and provided responsive dashboard output through a scalable web service architecture.

From the architectural perspective, the combination of EC2 for fog processing, AWS IoT Core for MQTT message ingestion, Lambdas for serverless event processing, and the DynamoDB for storage created a clean separation between edge functions and backend services. AWS IoT Core supports MQTT-based device communication, and Lambda provides event-driven execution without server management, making them suitable for a scalable IoT design. [10] [12]

One of the main findings from the implementation was that fog processing is valuable because it reduces direct dependency between raw sensor collection and the backend services. Instead of having sensors communicate directly with multiple cloud components, the fog node aggregates, formats, and publishes a consistent payload, while AWS IoT Core and its rules engine route that payload to the appropriate downstream services. [4]

Another important learning outcome was the practical advantage of serverless design for IoT backends. Lambda functions scale automatically with incoming events, and DynamoDB can be integrated programmatically through Boto3 operations such as put_item, making the backend easier to extend without manually provisioning compute resources.

A significant challenge during development was the retrieval of live noise data from websites. Because HTML structures can change over time, Beautiful Soup selectors are sensitive to layout modifications, which made the scraping logic less stable than the mock sensor components. Requests and Beautiful Soup are effective for downloading and parsing webpage content, but they require defensive coding and fallbacks when used in production-style pipelines. [9] [3] [2]

The dashboard implementation also demonstrated that not all processing must happen in the backend. The browser-side alert system used JavaScript to compare live readings against thresholds and update the page dynamically through DOM methods such as innerHTML, which simplified alert rendering while keeping the user interface responsive

Overall, this project provided a strong practical understanding of scalable IoT architecture, fog processing, MQTT messaging, serverless backends, and cloud-native dashboard delivery. The most important lesson learned was that a well-designed IoT system is not only about collecting data, but also about structuring the flow of data so that sensing, processing, storage, and presentation remain loosely coupled, scalable, and easier to maintain.

Future Scope:

- Code can be deployed to AWS Amplify instead of S3
- Add support for more realtime data sources and for multiple locations so the platform can compare conditions across the sites instead of monitoring a single stream
- Improve the dashboard with the live charts, filtering, and trend analysis so users can interpret the sensor conditions more effectively
- Extend the fog layer to support local decision making and the alerting before data reaches the cloud as well as reducing latency for the urgent events

References

[1] emqx, “Understanding AWS IoT Core: Features, Use Cases & Quick Tutorial,” emqx, 29 03 2026. [Online]. Available: https://www.emqx.com/en/blog/understanding-aws-iot-core. [Accessed 29 03 2026].

[2] K. Reitz, “Quickstart,” requests, 29 03 2026. [Online]. Available: https://requests.readthedocs.io/en/latest/user/quickstart/. [Accessed 29 03 2026].

[3] opensource, “Analyze web pages with Python requests and Beautiful Soup,” opensource, 29 03 2026. [Online]. Available: https://opensource.com/article/22/6/analyze-web-pages-python-requests-beautiful-soup. [Accessed 29 03 2026].

[4] AWS, “Rules for AWS IoT,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/iot-rules.html. [Accessed 29 03 2026].

[5] OneUptime, “How to Use IoT Core Rules Engine,” OneUptime, 29 03 2026. [Online]. Available: https://oneuptime.com/blog/post/2026-02-12-use-iot-core-rules-engine/view. [Accessed 29 03 2026].

[6] AWS, “How Lambda works,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/concepts-basics.html. [Accessed 29 03 2026].

[7] mozilla, “Element: innerHTML property,” mozilla, 29 03 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML. [Accessed 29 03 2026].

[8] AWS, “Boto3 documentation,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/boto3/latest/. [Accessed 29 03 2026].

[9] scraperapi, “BeautifulSoup’s Find and Find_All Method: Guide with Example,” scraperapi, 29 03 2026. [Online]. Available: https://www.scraperapi.com/blog/python-beautifulsoup-find-and-findall-methods/. [Accessed 29 03 2026].

[10] AWS, “MQTT,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html. [Accessed 29 03 2026].

[11] AWS, “Device communication protocols,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/protocols.html. [Accessed 29 03 2026].

[12] AWS, “What is AWS Lambda?,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html. [Accessed 29 03 2026].

[13] AWS, “Creating AWS IoT rules to route device data to other services,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/iot-rules-tutorial.html. [Accessed 29 03 2026].

[14] AWS, “Boto3 documentation,” AWS, 29 03 2026. [Online]. Available: https://docs.aws.amazon.com/boto3/latest/. [Accessed 29 03 2026].

[15] Gitleaks, “Gitleaks,” Gitleaks, 29 03 2026. [Online]. Available: https://gitleaks.io/. [Accessed 29 03 2026].

[16] Sonar, “Sonarqube docs,” Sonarqube, 29 03 2026. [Online]. Available: https://docs.sonarsource.com/. [Accessed 29 03 2026].